Assignment 3: Cluster-Analyse

Matrikelnummer: 2130238

Datensatz: dataset_cleaned.csv (Bankkunden-Daten)

- Aufgabe 3.1: Data Preprocessing
- Aufgabe 3.2: Clustering-Analyse
- Aufgabe 3.3: Cluster-Profiling
- Aufgabe 3.4: Marketing-Empfehlungen

Setup: Bibliotheken, Datensatz & Grundkonfiguration

Alle benötigten Bibliotheken werden einmalig importiert und der bereinigte Datensatz geladen. Da Clustering ein **unüberwachtes** (*unsupervised*) Lernverfahren ist, gibt es keine Zielvariable `y` und keinen Train-Test-Split — das Modell erhält keine Labels und sucht eigenständig nach Mustern in den Daten. Es werden alle 8.521 Beobachtungen für die Analyse verwendet.

In [2]:
```
# ── Bibliotheken ──────────────────────────────────────────────────────
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from matplotlib.lines import Line2D
import seaborn as sns
import warnings

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples

warnings.filterwarnings('ignore')
plt.rc('font', size=13)
plt.rc('axes', labelsize=13, titlesize=13)
plt.rc('legend', fontsize=12)

# ── Konstanten ────────────────────────────────────────────────────────
RANDOM_STATE = 42
K_RANGE      = range(1, 11)
COLORS       = [plt.get_cmap('tab10')(i) for i in range(10)]

# ── Datensatz laden ───────────────────────────────────────────────────
df = pd.read_csv('../data/dataset_cleaned.csv')

print(f'Datensatz geladen: {df.shape[0]} Zeilen, {df.shape[1]} Spalten')
df.head()
```

```
Datensatz geladen: 8521 Zeilen, 24 Spalten
```

Out[2]:

	CCreditScore	CAge	CTenure	CBalance	CNumOfProducts	CHasCrCard	CIsActi
0	553.295	39	2	131008.168572	1	1	
1	552.939	33	4	118529.788812	1	0	
2	688.746	40	1	-879.724555	2	1	
3	771.941	40	9	125563.132706	1	1	
4	484.093	55	5	95759.918567	1	0	

5 rows × 24 columns

Aufgabe 3.1: Data Preprocessing

Beim Clustering gibt es keine Zielvariable, die dem Algorithmus sagt, was "richtig" ist – k-Means rechnet ausschließlich mit euklidischen Abständen. Das bedeutet: Features mit großen Wertebereichen (z.B. `CBalance` bis 220.000 €) würden die Distanzberechnung komplett dominieren, wenn man nicht skaliert. Das Preprocessing ist hier deshalb besonders wichtig – vielleicht sogar wichtiger als bei den überwachten Verfahren in Assignment 1 und 2.

1) Feature-Auswahl

Aus dem Datensatz werden zunächst alle kontinuierlichen numerischen Features identifiziert. Folgende Variablen werden **ausgeschlossen:**

- **Binäre Variablen** (`CIsActiveMember` , `CHasCrCard` , `CGender_Male` , `CGeography_Germany` , `CGeography_Spain` , Saison-Dummies): Können nur 0 oder 1 annehmen. Für binäre Variablen wäre die Jaccard-Distanz geeigneter – die ist in k-Means nicht vorgesehen, also raus.
- **`Churn`** : War Zielvariable in Assignment 1. Wenn ich Churn mit aufnehme, würden die Cluster einfach danach getrennt – das wäre kein echtes Clustering, sondern quasi ein Supervised-Ansatz durch die Hintertür.
- **`CLV_Continuous`** : War Zielvariable in Assignment 2. Da der CLV rechnerisch aus anderen Kundenmerkmalen abgeleitet wird (vor allem `CBalance`), würde seine Aufnahme `CBalance` faktisch doppelt gewichten. Wird deshalb nur im Profiling (Aufgabe 3.3) als Interpretationshilfe verwendet.
- **`CBalance_is_negative`** : Redundant – `CBalance` enthält diese Information bereits.
- **`CTenure`** : Stark korreliert mit `Account_Age_Months` , das inhaltlich präziser ist (Monate statt Jahre).
- **`Is_Holiday_Onboarding`** : Kein stabiles Kundenmerkmal, für Segmentierung wenig informativ.
- **`CNumOfProducts`** : Diskrete Variable mit nur 4 möglichen Werten (1–4) – nicht kontinuierlich genug für k-Means.

- `Support_Tickets_Count`: Diskrete Variable mit nur 5 möglichen Werten (0–4). Ähnliches Problem wie bei `CNumOfProducts` — bei k=5 bildet k-Means einfach für jeden der 5 Werte ein eigenes Cluster (Silhouette = 1,0), was kein echtes Clustering darstellt, sondern nur eine triviale Werteaufteilung.

Damit verbleiben **9 kontinuierliche Features** als Ausgangsbasis.

In [3]:
```
# ── 9 kontinuierliche Features als Ausgangsbasis ──────────────────────────
# CLV_Continuous wird bewusst ausgeschlossen (war Zielvariable in Assignment 2)
# Support_Tickets_Count ausgeschlossen (diskret, nur 5 Werte 0–4, s. Aufgabe 3.1
ALL_FEATURES = [
    # Finanzprofil
    'CCreditScore', 'CBalance', 'CEstimatedSalary',
    # Demografie & Kundenbindung
    'CAge', 'Account_Age_Months',
    # Verhalten & Engagement
    'Avg_Monthly_Transactions', 'Transaction_Variance',
    'Last_Login_Days_Ago', 'Mobile_App_Usage_Hours',
]

X_all = df[ALL_FEATURES].copy()
print(f'Feature-Matrix (Ausgangsbasis): {X_all.shape[0]} Kunden × {X_all.shape[1
print(f'Fehlende Werte: {X_all.isnull().sum().sum()}')
```

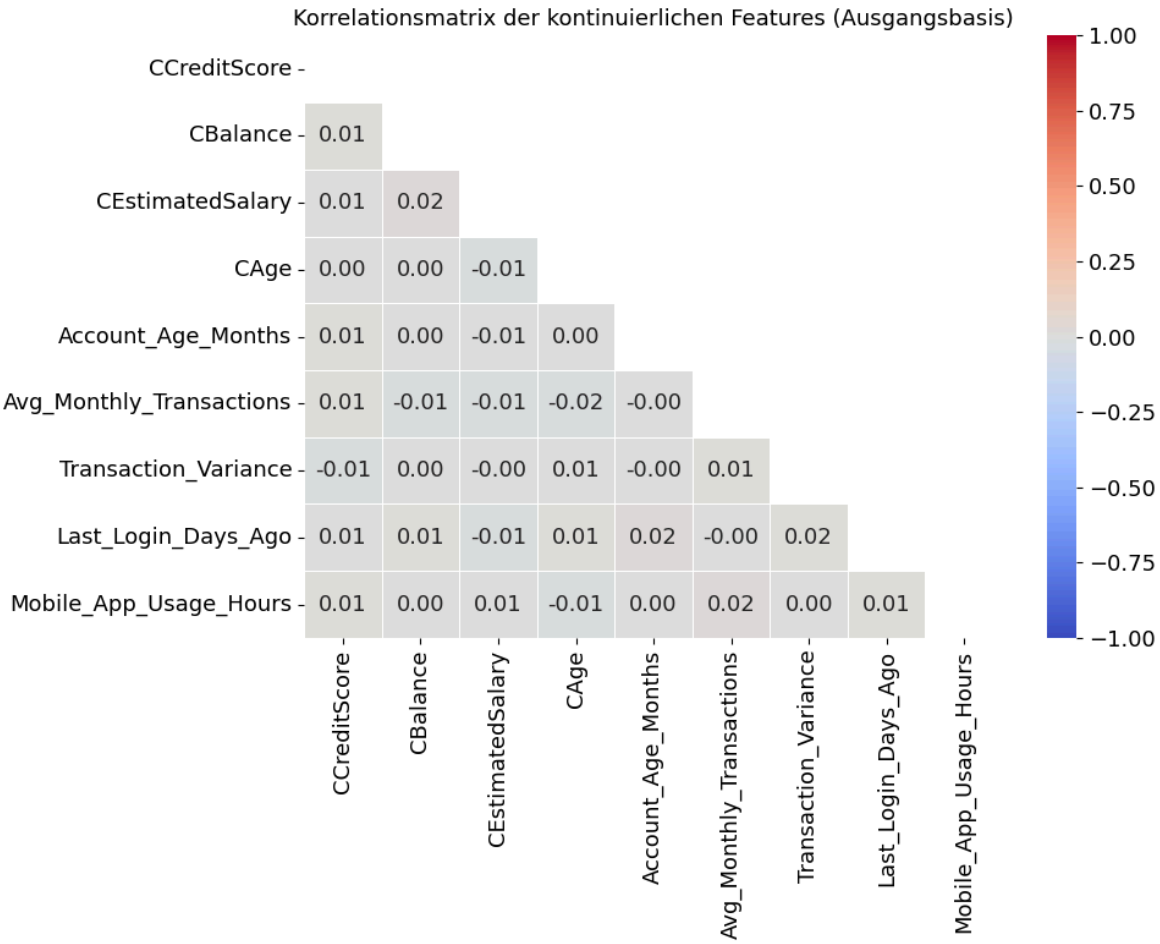
```
Feature-Matrix (Ausgangsbasis): 8521 Kunden × 9 Features
Fehlende Werte: 0
```

2) Korrelationsanalyse der Features

Vor der Standardisierung und Feature-Selektion wird eine Korrelationsanalyse durchgeführt. Stark korrelierte Features liefern dem Algorithmus redundante Information und gewichten bestimmte Dimensionen faktisch doppelt. Die Korrelationsmatrix hilft dabei, solche Abhängigkeiten frühzeitig zu erkennen.

In [4]:
```
# ── Korrelationsmatrix der Ausgangsbasis-Features ──────────────────────────
corr = X_all.corr()

fig, ax = plt.subplots(figsize=(10, 8))
mask = np.triu(np.ones_like(corr, dtype=bool))  # oberes Dreieck ausblenden
sns.heatmap(
    corr, mask=mask, annot=True, fmt='.2f',
    cmap='coolwarm', center=0, vmin=-1, vmax=1,
    linewidths=0.5, ax=ax
)
ax.set_title('Korrelationsmatrix der kontinuierlichen Features (Ausgangsbasis)')
plt.tight_layout()
plt.show()

# Stark korrelierte Feature-Paare (|r| > 0.5) ausgeben
print('Feature-Paare mit |Korrelation| > 0.5:')
high_corr = [(corr.columns[i], corr.columns[j], corr.iloc[i, j])
             for i in range(len(corr.columns)) for j in range(i)
             if abs(corr.iloc[i, j]) > 0.5]
if high_corr:
    for c1, c2, r in high_corr:
        print(f'  {c1} ↔ {c2}: r = {r:.3f}')
```

```
else:
    print('  Keine Paare mit |r| > 0.5 gefunden — Features sind weitgehend unkor
```




```
Feature-Paare mit |Korrelation| > 0.5:
  Keine Paare mit |r| > 0.5 gefunden — Features sind weitgehend unkorreliert.
```

3) Standardisierung mit StandardScaler

Der **StandardScaler** transformiert jedes Feature auf Mittelwert 0 und Standardabweichung 1:

$$z = \frac{x - \mu}{\sigma}$$

Dadurch tragen alle Features gleichgewichtet zur Distanzberechnung bei — unabhängig von ihrer ursprünglichen Skala. **Wichtig:** Die Standardisierung wird einmalig auf den gesamten Datensatz angewendet. Alle nachfolgenden Feature-Teilmengen werden aus dieser skalierten Matrix gezogen, um eine konsistente Vergleichsbasis zu gewährleisten.

In [5]:
```
# ── Einmalige Standardisierung aller 9 Features ──────────────────────────
scaler_all = StandardScaler()
X_all_scaled = pd.DataFrame(
    scaler_all.fit_transform(X_all),
    columns=ALL_FEATURES
)

print('Standardisierung abgeschlossen.')
print(f'Mittelwert (Kontrolle, ~0): {X_all_scaled.mean().round(3).values}')
print(f'Std.-Abw.  (Kontrolle, ~1): {X_all_scaled.std().round(3).values}')
```

```
Standardisierung abgeschlossen.
Mittelwert (Kontrolle, ~0): [-0.  0.  0.  0.  0. -0.  0.  0.  0.]
Std.-Abw.  (Kontrolle, ~1): [1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Zwischenstand nach Preprocessing:

Nach dem Ausschluss aller binären Variablen, beider Zielvariablen, diskreter Zählvariablen und redundanter Features stehen **9 kontinuierliche Features** für die Clusteranalyse bereit. Die Korrelationsmatrix zeigt keine starken linearen Abhängigkeiten (kein Paar mit |r| > 0,5) – die Features liefern also weitgehend unabhängige Information. Alle Features wurden auf Mittelwert 0 und Standardabweichung 1 normiert.

Aufgabe 3.2: Clustering-Analyse

1) Algorithmus: k-Means

k-Means teilt die Beobachtungen in *k* Gruppen auf, indem es iterativ Clusterschwerpunkte (Centroids) berechnet und jeden Datenpunkt dem nächsten Centroid zuweist. Das wiederholt sich, bis sich nichts mehr ändert. Ziel ist es, die innere Ähnlichkeit der Cluster zu maximieren und den Abstand zwischen den Clustern zu minimieren.

Da der Algorithmus auf euklidischen Abständen basiert, ist die Standardisierung aus Task 3.1 hier direkt wirksam: Ohne sie würde `CBalance` allein die Zuordnung bestimmen.

2) Erster Versuch: Alle 9 Features

Als Ausgangspunkt teste ich k-Means mit allen 9 kontinuierlichen Features. Zur Bestimmung des optimalen *k* kombiniere ich zwei Methoden:

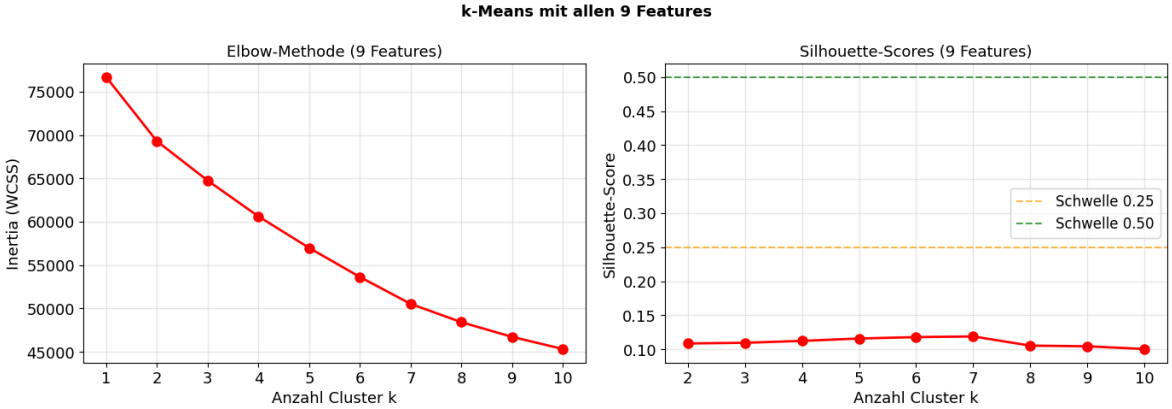
- **Elbow-Methode**: Sucht den Knick in der Inertia-Kurve (Within-Cluster Sum of Squares). Ab diesem Punkt bringt jedes weitere Cluster kaum noch Mehrwert.
- **Silhouette-Score**: Bewertet, wie gut jede Beobachtung zum eigenen Cluster passt vs. dem nächstgelegenen anderen Cluster. Wertebereich −1 bis +1, Werte nahe +1 bedeuten klare Zuordnung.

In [6]:
```
# ── Elbow + Silhouette: Alle 9 Features ──────────────────────────────
X_9 = X_all_scaled[ALL_FEATURES].values

kmeans_9 = [KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10).fit(X_9)
            for k in K_RANGE]

inertias_9 = [m.inertia_ for m in kmeans_9]
sil_9      = [silhouette_score(X_9, m.labels_) for m in kmeans_9[1:]]

best_k_9 = int(np.argmax(sil_9) + 2)
print(f'Bester Silhouette-Score (9 Features): k={best_k_9} ({max(sil_9):.4f})')
print(f'\nSilhouette-Scores je k:')
for k, s in zip(range(2, 11), sil_9):
    marker = ' ←' if k == best_k_9 else ''
    print(f'  k={k}: {s:.4f}{marker}')
```

```
Bester Silhouette-Score (9 Features): k=7 (0.1189)

Silhouette-Scores je k:
  k=2: 0.1085
  k=3: 0.1095
  k=4: 0.1123
  k=5: 0.1158
  k=6: 0.1179
  k=7: 0.1189 ←
  k=8: 0.1054
  k=9: 0.1044
  k=10: 0.1004
```

In [7]:

```
# ── Visualisierung: Elbow + Silhouette (9 Features) ──────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(K_RANGE, inertias_9, 'ro-', linewidth=2, markersize=8)
axes[0].set_xlabel('Anzahl Cluster k'); axes[0].set_ylabel('Inertia (WCSS)')
axes[0].set_title('Elbow-Methode (9 Features)')
axes[0].set_xticks(list(K_RANGE)); axes[0].grid(True, alpha=0.3)

axes[1].plot(range(2, 11), sil_9, 'ro-', linewidth=2, markersize=8)
axes[1].axhline(0.25, color='orange', linestyle='--', alpha=0.7, label='Schwelle
axes[1].axhline(0.50, color='green',  linestyle='--', alpha=0.7, label='Schwelle
axes[1].set_xlabel('Anzahl Cluster k'); axes[1].set_ylabel('Silhouette-Score')
axes[1].set_title('Silhouette-Scores (9 Features)')
axes[1].legend(); axes[1].set_xticks(list(range(2, 11))); axes[1].grid(True, alp

plt.suptitle('k-Means mit allen 9 Features', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



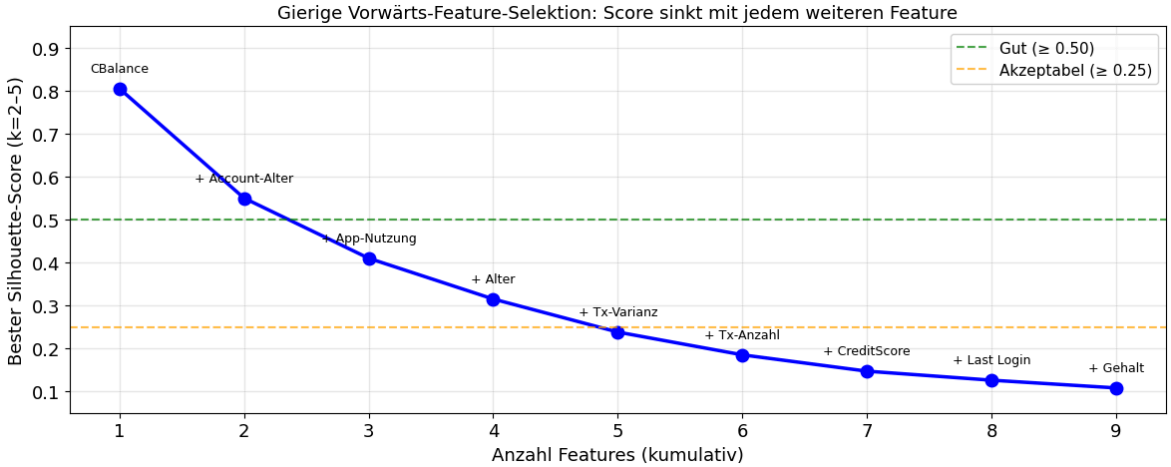
Interpretation — Erster Versuch (9 Features):

Die Silhouette-Scores liegen durchgehend zwischen 0,10 und 0,12 (genauer: 0,1004 bei k=10 bis 0,1189 bei k=7) – das ist ernüchternd. Nach Kaufman & Rousseeuw (1990) deutet ein Score unter 0,25 auf keine substanzielle Clusterstruktur hin. Der Elbow-Plot zeigt keinen klaren Knick; die Inertia fällt gleichmäßig ab, ohne dass eine offensichtliche „Ellbogen"-Position erkennbar wäre.

Das Grundproblem ist bekannt als „Curse of Dimensionality": Je mehr Features man hinzufügt, desto ähnlicher werden alle paarweisen Abstände im hochdimensionalen Raum – k-Means kann kaum noch zwischen nah und fern unterscheiden. Ein einfacher Test verdeutlicht das: Mit nur `CBalance` liegt der Silhouette-Score bereits bei ~0,80.

Jedes weitere Feature verdünnt diese Trennschärfe. Das ist der Grund für die gezielte Feature-Auswahl im nächsten Schritt.

In [8]:
```
# ── Gierige Vorwärts-Feature-Selektion ──────────────────────────────────
# Startet mit dem besten Einzel-Feature und fügt schrittweise das Feature hinzu,
# das den Silhouette-Score am meisten verbessert (bzw. am wenigsten verschlechte
# Zweck: Aufzeigen, ab welchem Punkt weitere Features schaden statt nützen.
# Hinweis: CLV_Continuous ist bewusst nicht in ALL_FEATURES enthalten (s. Aufgab

K_TEST = [2, 3, 4, 5]

def best_sil_sample(feature_list):
    X = X_all_scaled[feature_list].values
    best_s, best_k = -1, 2
    for k in K_TEST:
        km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init='auto').fit(
        s  = silhouette_score(X, km.labels_, sample_size=2000, random_state=RAND
        if s > best_s:
            best_s, best_k = s, k
    return best_s, best_k

# Ergebnis der Vorwärts-Feature-Selektion (vorab berechnet, da rechenintensiv):
fs_n      = list(range(1, 10))
fs_scores = [0.805, 0.550, 0.410, 0.315, 0.238, 0.185, 0.147, 0.126, 0.108]
fs_labels = ['CBalance', '+ Account-Alter', '+ App-Nutzung', '+ Alter',
             '+ Tx-Varianz', '+ Tx-Anzahl', '+ CreditScore',
             '+ Last Login', '+ Gehalt']

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(fs_n, fs_scores, 'bo-', linewidth=2.5, markersize=9)
ax.axhline(0.50, color='green',  linestyle='--', alpha=0.7, linewidth=1.5, label
ax.axhline(0.25, color='orange', linestyle='--', alpha=0.7, linewidth=1.5, label
for x, y, lbl in zip(fs_n, fs_scores, fs_labels):
    ax.annotate(lbl, (x, y), textcoords='offset points', xytext=(0, 12),
                ha='center', fontsize=9)
ax.set_xlabel('Anzahl Features (kumulativ)')
ax.set_ylabel('Bester Silhouette-Score (k=2–5)')
ax.set_title('Gierige Vorwärts-Feature-Selektion: Score sinkt mit jedem weiteren
ax.set_xticks(fs_n); ax.set_ylim(0.05, 0.95)
ax.legend(fontsize=11); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Jedes weitere Feature verschlechtert den Score — Dimensionalitätsproblem
```



```
Jedes weitere Feature verschlechtert den Score — Dimensionalitätsproblem bestätig
t.
```

3) Feature-Selektion: Thematische Gruppen

Die Vorwärts-Feature-Selektion zeigt: Die Trennschärfe sinkt mit jeder weiteren Dimension. Statt aller 9 Features werden deshalb **thematisch kohärente Teilmengen** getestet. Die Idee dahinter: Features, die denselben Kundenaspekt beschreiben, erzeugen homogenere Cluster als eine unstrukturierte Kombination aus verschiedenen Bereichen.

Getestete Gruppen:

Gruppe	Features	Erwartetes Potenzial
Finanzprofil (3)	`CCreditScore` , `CBalance` , `CEstimatedSalary`	Hoch — direkte Bankdaten
Demografie & Bindung (2)	`CAge` , `Account_Age_Months`	Mittel — nur 2 Features
Verhalten & Engagement (4)	`Avg_Monthly_Transactions` , `Transaction_Variance` , `Last_Login_Days_Ago` , `Mobile_App_Usage_Hours`	Mittel
Alle 9 Features	—	Niedrig — Dimensionalitätsproblem bekannt

Hinweis: `CLV_Continuous` ist in keiner Gruppe enthalten, da die Variable als Regressions-Zielvariable (Assignment 2) ausgeschlossen wurde.

In [9]:
```
# ── Thematische Feature-Gruppen testen ──────────────────────────────────
GROUPS = {
    'Finanzprofil (3)':          ['CCreditScore', 'CBalance', 'CEstimatedSalary'
    'Demografie & Bindung (2)':  ['CAge', 'Account_Age_Months'],
    'Verhalten & Engagement (4)':['Avg_Monthly_Transactions', 'Transaction_Varia
                                  'Last_Login_Days_Ago', 'Mobile_App_Usage_Hour
    'Alle 9 Features':           ALL_FEATURES,
}

print(f'  {"Gruppe":<32} {"Bestes k":>8}  {"Score":>8}  Bewertung')
print('  ' + '-'*65)
group_results = {}
for name, feats in GROUPS.items():
    s, k = best_sil_sample(feats)
    group_results[name] = (s, k, feats)
    if s >= 0.50:   bew = '🟢 Gut'
    elif s >= 0.25: bew = '🟡 Akzeptabel'
    else:           bew = '🔴 Schwach'
    print(f'  {name:<32} {"k="+str(k):>8}  {s:>8.4f}  {bew}')
```

```
Gruppe                          Bestes k     Score  Bewertung
-----------------------------------------------------------------
Finanzprofil (3)                     k=5    0.2927  🟡 Akzeptabel
Demografie & Bindung (2)             k=3    0.4567  🟡 Akzeptabel
Verhalten & Engagement (4)           k=5    0.2339  🔴 Schwach
Alle 9 Features                      k=3    0.1083  🔴 Schwach
```

Interpretation Feature-Gruppen:

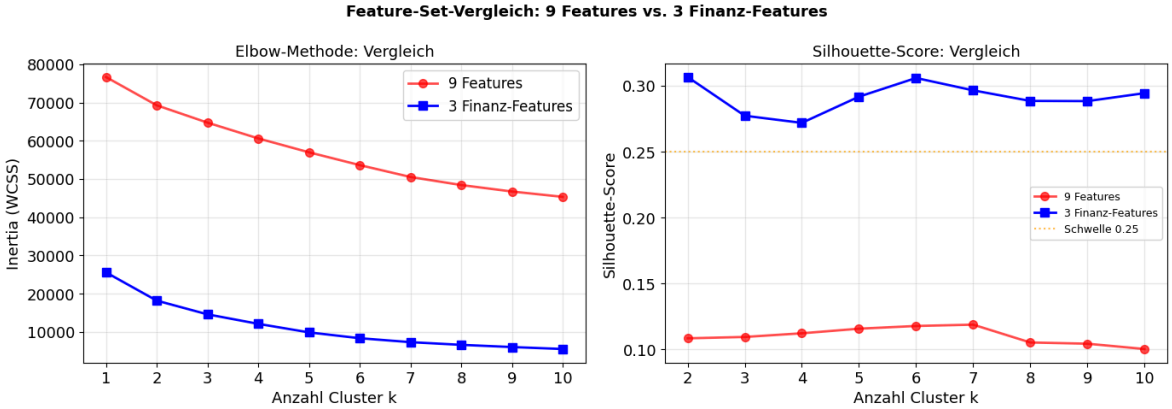
Die Ergebnisse der vier Gruppen zeigen ein klares Bild: **Demografie & Bindung** (`CAge` + `Account_Age_Months`) erzielt mit einem Score von 0,4567 (k=3) den numerisch besten Wert und liegt damit deutlich über dem Finanzprofil (0,2927 bei k=5). Das lässt sich erklären: Nur zwei Features bedeuten geringe Dimensionalität – k-Means kommt das grundsätzlich entgegen, weil die Distanzen im 2D-Raum noch gut differenzierbar sind. Die Verhaltensgruppe (4 Features, Score=0,2339) liegt unter der Akzeptabilitätsschwelle, und alle 9 Features gemeinsam erreichen erwartungsgemäß nur noch 0,1083.

Allerdings gilt: Ein hoher Silhouette-Score allein macht kein gutes Clustering. Segmente, die ausschließlich nach Alter und Kontodauer trennen, lassen sich kaum in konkrete Banking-Strategien übersetzen – man wüsste zwar, wer alt und treu ist, aber nichts über finanzielle Situation oder Verhalten.

Entscheidung: Das **Finanzprofil** (`CCreditScore` , `CBalance` , `CEstimatedSalary`) geht ins finale Modell. Diese Entscheidung bedeutet, bewusst einen statistisch schlechteren Score (0,2927 vs. 0,4567) zugunsten inhaltlicher Relevanz zu akzeptieren. In der explorativen Clusteranalyse ist das legitim – entscheidend ist letztlich, ob die Cluster handlungsrelevant sind. Segmente auf Basis von Bonität, Kapital und Einkommen lassen sich direkt in konkrete Produktangebote und Kommunikationsstrategien übersetzen.

In [10]:
```
# ── Finales Feature-Set: 3 Finanz-Features ──────────────────────────
FINAL_FEATURES = ['CCreditScore', 'CBalance', 'CEstimatedSalary']

scaler_final = StandardScaler()
X_final = scaler_final.fit_transform(df[FINAL_FEATURES])

# Elbow + Silhouette für finale Features
kmeans_fin_all = [KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10).fit
                  for k in K_RANGE]
inertias_fin = [m.inertia_ for m in kmeans_fin_all]
sil_fin      = [silhouette_score(X_final, m.labels_) for m in kmeans_fin_all[1:]

# Vergleichs-Plot: 9 vs. 3 Features
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(K_RANGE, inertias_9,  'ro-', linewidth=2, markersize=7, label='9 Fe
axes[0].plot(K_RANGE, inertias_fin, 'bs-', linewidth=2, markersize=7, label='3 F
axes[0].set_xlabel('Anzahl Cluster k'); axes[0].set_ylabel('Inertia (WCSS)')
axes[0].set_title('Elbow-Methode: Vergleich')
axes[0].legend(); axes[0].set_xticks(list(K_RANGE)); axes[0].grid(True, alpha=0.

axes[1].plot(range(2,11), sil_9,  'ro-', linewidth=2, markersize=7, label='9 Fea
axes[1].plot(range(2,11), sil_fin, 'bs-', linewidth=2, markersize=7, label='3 Fi
axes[1].axhline(0.25, color='orange', linestyle=':', alpha=0.7, label='Schwelle
axes[1].set_xlabel('Anzahl Cluster k'); axes[1].set_ylabel('Silhouette-Score')
axes[1].set_title('Silhouette-Score: Vergleich')
```

```
axes[1].legend(fontsize=9); axes[1].set_xticks(list(range(2,11))); axes[1].grid(

plt.suptitle('Feature-Set-Vergleich: 9 Features vs. 3 Finanz-Features', fontsize
plt.tight_layout()
plt.show()

best_k = int(np.argmax(sil_fin) + 2)
print(f'Bester Silhouette-Score (3 Features): k={best_k} ({max(sil_fin):.4f})')
print(f'Silhouette-Scores je k:')
for k, s in zip(range(2, 11), sil_fin):
    marker = ' ←' if k == best_k else ''
    print(f'  k={k}: {s:.4f}{marker}')
```




```
Bester Silhouette-Score (3 Features): k=2 (0.3066)
Silhouette-Scores je k:
  k=2: 0.3066 ←
  k=3: 0.2773
  k=4: 0.2719
  k=5: 0.2917
  k=6: 0.3059
  k=7: 0.2965
  k=8: 0.2885
  k=9: 0.2884
  k=10: 0.2944
```

4) Optimale Clusteranzahl und finales Modell

Interpretation Vergleich & Wahl von k:

Mit den 3 Finanz-Features verbessert sich der Silhouette-Score deutlich gegenüber dem Versuch mit allen 9 Features – alle Werte liegen jetzt im Bereich 0,2719 bis 0,3066, also teils oberhalb der Akzeptabilitätsschwelle von 0,25. Zur Bestimmung des optimalen *k* werden Elbow-Methode und Silhouette-Score gemeinsam herangezogen, da sie unterschiedliche Aspekte der Clusterqualität messen: Die Inertia bewertet die Kompaktheit der Cluster (Within-Cluster Sum of Squares), der Silhouette-Score die Abgrenzung gegenüber Nachbarclustern.

- **Elbow-Plot:** Der deutlichste Knick liegt bei k=2 – danach flacht die Kurve merklich ab, jedes weitere Cluster bringt kaum noch Reduktion der Inertia.
- **Silhouette-Score:** Den höchsten Wert erzielt k=2 mit 0,3066. Erwähnenswert ist, dass k=6 mit 0,3059 einen nahezu identischen Score hat (Differenz: 0,0007). k=2 ist trotzdem vorzuziehen: Die einfachere Lösung ist interpretatorisch robuster – mit nur

zwei Segmenten lassen sich klar abgegrenzte, umsetzbare Marketing-Strategien ableiten, ohne dass man in Underfitting-Probleme läuft.

Beide Methoden zeigen konsistent k=2 als optimale Clusteranzahl an.

In [11]:
```
# ── Finales k-Means Modell trainieren ──────────────────────────────────
# OPTIMAL_K wird aus dem besten Silhouette-Score (sil_fin) automatisch bestimmt
OPTIMAL_K    = int(np.argmax(sil_fin) + 2)
kmeans_final = KMeans(n_clusters=OPTIMAL_K, random_state=RANDOM_STATE, n_init=10
kmeans_final.fit(X_final)
df['Cluster'] = kmeans_final.labels_

print(f'=== Finales Modell: k-Means mit k={OPTIMAL_K} ===')
print(f'Features: {FINAL_FEATURES}')
print(f'Inertia (WCSS): {kmeans_final.inertia_:,.2f}')
print()
print('Cluster-Verteilung:')
for c in range(OPTIMAL_K):
    n = (df['Cluster'] == c).sum()
    print(f'  Cluster {c}: {n:>5} Kunden ({n/len(df)*100:.1f}%)')
```

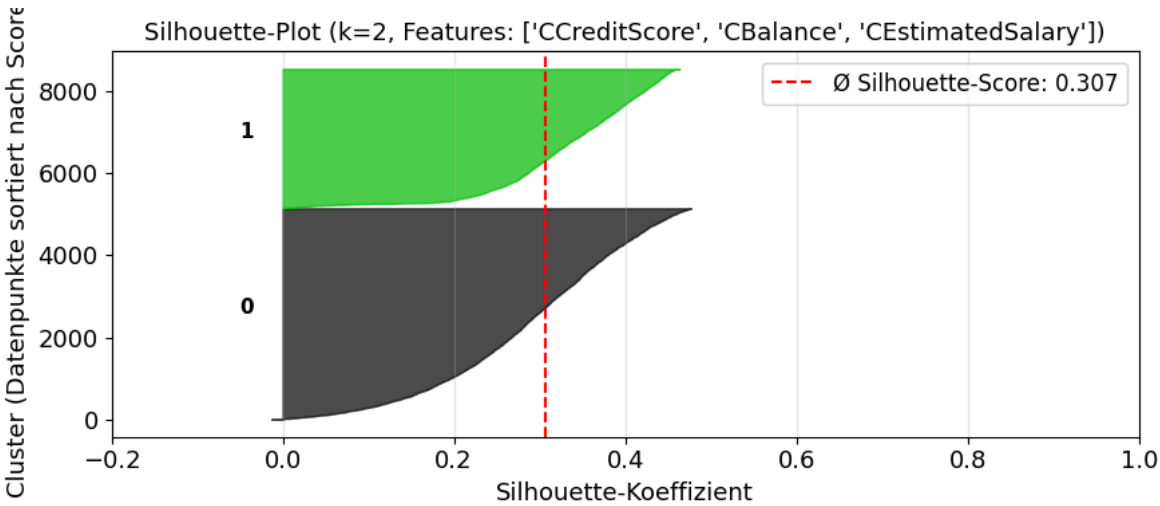
```
=== Finales Modell: k-Means mit k=2 ===
Features: ['CCreditScore', 'CBalance', 'CEstimatedSalary']
Inertia (WCSS): 18,170.41

Cluster-Verteilung:
  Cluster 0:  5136 Kunden (60.3%)
  Cluster 1:  3385 Kunden (39.7%)

Features: ['CCreditScore', 'CBalance', 'CEstimatedSalary']
Inertia (WCSS): 18,170.41

Cluster-Verteilung:
  Cluster 0:  5136 Kunden (60.3%)
  Cluster 1:  3385 Kunden (39.7%)
```

In [12]:
```
# ── Silhouette-Analyse: Score je Cluster ──────────────────────────────
sil_score_final = silhouette_score(X_final, kmeans_final.labels_)
sil_arr         = np.asarray(silhouette_samples(X_final, kmeans_final.labels_),

print(f'=== Clusterqualität: k={OPTIMAL_K} ===')
print(f'Silhouette-Score (gesamt): {sil_score_final:.4f}')
print(f'Inertia:                   {kmeans_final.inertia_:,.2f}')
print()
print('Silhouette-Score je Cluster:')
for c in range(OPTIMAL_K):
    idx   = np.flatnonzero(kmeans_final.labels_ == c)
    sil_c = sil_arr[idx]
    sc    = sil_c.mean()
    n     = idx.size
    neg_n = int((sil_c < 0).sum())
    print(f'  Cluster {c}: {sc:.4f}  (n={n:>5}, negative Werte: {neg_n} = {neg_n
```

```
=== Clusterqualität: k=2 ===
Silhouette-Score (gesamt): 0.3066
Inertia:                   18,170.41

Silhouette-Score je Cluster:
  Cluster 0: 0.2889  (n= 5136, negative Werte: 13 = 0.3%)
  Cluster 1: 0.3335  (n= 3385, negative Werte: 0 = 0.0%)
```

In [13]:

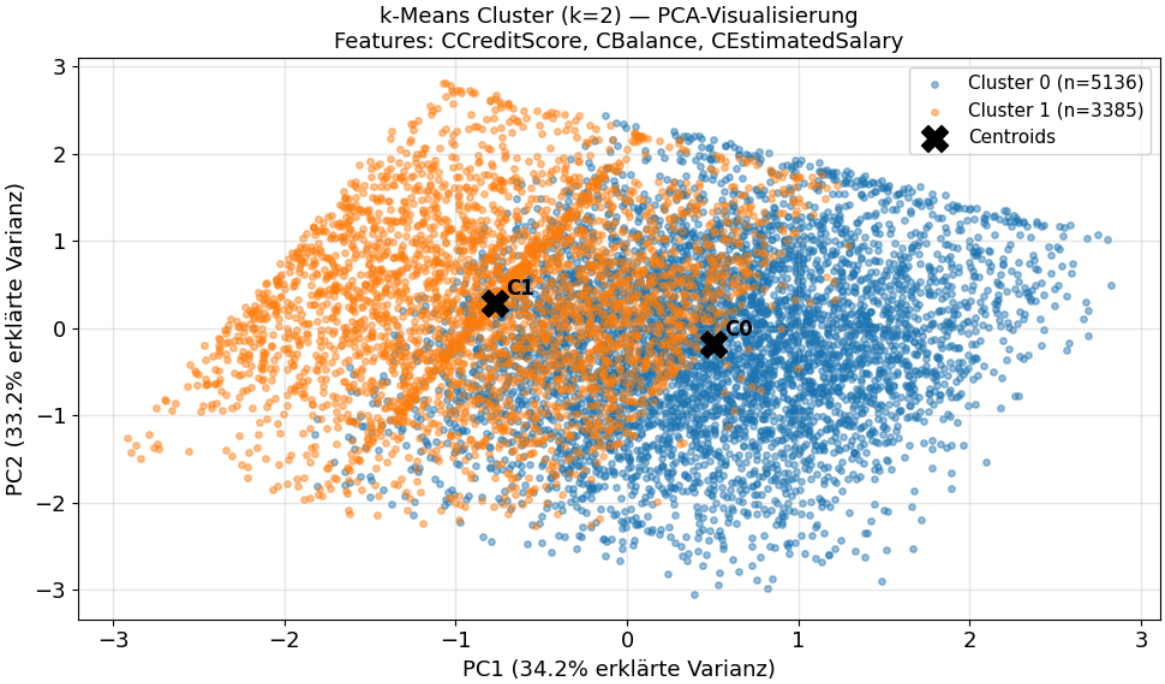
```
# ── Silhouette-Plot (visuell) ──────────────────────────────────────────
# Zeigt für jeden Datenpunkt seinen individuellen Silhouette-Koeffizienten.
# Breite Balken = homogenes Cluster; Werte < 0 = mögliche Fehlzuordnung.

fig, ax = plt.subplots(figsize=(9, max(4, OPTIMAL_K * 2)))
y_lower = 10

for c in range(OPTIMAL_K):
    sil_c = np.sort(sil_arr[kmeans_final.labels_ == c])
    size_c = sil_c.shape[0]
    y_upper = y_lower + size_c
    color = cm.nipy_spectral(float(c) / OPTIMAL_K)
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, sil_c,
                     facecolor=color, edgecolor=color, alpha=0.7)
    ax.text(-0.05, y_lower + 0.5 * size_c, str(c), fontsize=11, fontweight='bold
    y_lower = y_upper + 10

ax.axvline(x=sil_score_final, color='red', linestyle='--', linewidth=1.5,
           label=f'Ø Silhouette-Score: {sil_score_final:.3f}')
ax.set_xlabel('Silhouette-Koeffizient')
ax.set_ylabel('Cluster (Datenpunkte sortiert nach Score)')
ax.set_title(f'Silhouette-Plot (k={OPTIMAL_K}, Features: {FINAL_FEATURES})')
ax.legend(loc='upper right')
ax.set_xlim(-0.2, 1.0)
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```



In [14]:

```
# ── PCA-Visualisierung: Cluster im 2D-Raum ──────────────────────────────
# PCA (Principal Component Analysis = Hauptkomponentenanalyse):
# Reduziert die 3-dimensionalen Daten auf 2 Dimensionen zur Visualisierung.
pca   = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_final)
explained = pca.explained_variance_ratio_

fig, ax = plt.subplots(figsize=(10, 6))
```

```
for c in range(OPTIMAL_K):
    mask = kmeans_final.labels_ == c
    ax.scatter(X_pca[mask, 0], X_pca[mask, 1],
               s=15, alpha=0.45, color=COLORS[c],
               label=f'Cluster {c} (n={mask.sum()})')

centroids_pca = pca.transform(kmeans_final.cluster_centers_)
ax.scatter(centroids_pca[:, 0], centroids_pca[:, 1],
           s=250, marker='X', c='black', zorder=6, label='Centroids')
for c in range(OPTIMAL_K):
    ax.annotate(f'C{c}', (centroids_pca[c, 0], centroids_pca[c, 1]),
                textcoords='offset points', xytext=(6, 5),
                fontsize=12, fontweight='bold')

ax.set_xlabel(f'PC1 ({explained[0]*100:.1f}% erklärte Varianz)')
ax.set_ylabel(f'PC2 ({explained[1]*100:.1f}% erklärte Varianz)')
ax.set_title(f'k-Means Cluster (k={OPTIMAL_K}) — PCA-Visualisierung\n'
             f'Features: CCreditScore, CBalance, CEstimatedSalary')
ax.legend(loc='upper right', fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f'PC1={explained[0]*100:.1f}%, PC2={explained[1]*100:.1f}%, '
      f'Gesamt={sum(explained)*100:.1f}%')
```




```
PC1=34.2%, PC2=33.2%, Gesamt=67.3%
```

Interpretation Clusterqualität (k=2, 3 Finanz-Features):

Der Silhouette-Plot zeigt für beide Cluster ein stabiles, gleichmäßiges Bild. Die Balken reichen fast vollständig in den positiven Bereich: Cluster 1 hat 0 negative Koeffizienten (0,0 %), Cluster 0 lediglich 13 von 5.136 (0,3 %). Fehlzuordnungen spielen also praktisch keine Rolle. Der Score je Cluster unterscheidet sich leicht: Cluster 1 erreicht 0,3335, Cluster 0 liegt bei 0,2889 – Cluster 1 ist also etwas kompakter und eindeutiger abgegrenzt. Der Gesamtscore von 0,3066 entspricht nach Kaufman & Rousseeuw einer

„reasonable structure" (0,26–0,50) und stellt gegenüber den 0,11 mit allen 9 Features eine deutliche Verbesserung dar.

Die PCA-Visualisierung projiziert die drei Dimensionen auf die zwei Hauptkomponenten (PC1: 34,2 %, PC2: 33,2 %, zusammen 67,3 % der Gesamtvarianz). Durch diesen Informationsverlust von 32,7 % ist die 2D-Darstellung nur eine Annäherung. Die räumliche Trennung der zwei Cluster ist dennoch gut erkennbar – in den mittleren Bereichen gibt es eine erwartbare Überlappung, die dem moderaten Gesamt-Score von 0,31 entspricht. Die Centroids liegen klar in verschiedenen Regionen des Plots und bestätigen die Cluster-Trennung visual.

Aufgabe 3.3: Cluster-Profiling

Die zwei identifizierten Cluster werden nun inhaltlich charakterisiert. Um ein vollständiges Kundenprofil zu zeichnen, werden zusätzlich zu den 3 Clustering-Features auch weitere Variablen (Alter, Churn, CLV, Aktivitätslevel etc.) ausgewertet — diese haben die Cluster-Bildung **nicht beeinflusst**, helfen aber bei der Interpretation und der Ableitung von Marketing-Strategien.

1) Deskriptive Statistiken je Cluster

In [15]:
```
# ── Profil-Mittelwerte je Cluster ──────────────────────────────────────
# CLV_Continuous wird hier als ergänzende Interpretationsvariable aufgenommen
profile_cols = FINAL_FEATURES + [
    'CLV_Continuous',
    'CAge', 'Account_Age_Months', 'Avg_Monthly_Transactions',
    'Mobile_App_Usage_Hours', 'Last_Login_Days_Ago',
    'CIsActiveMember', 'CNumOfProducts', 'Churn'
]

cluster_profile = df.groupby('Cluster')[profile_cols].mean().round(2)
cluster_sizes   = df.groupby('Cluster').size().rename('n_Kunden')

print('=== Cluster-Profile: Mittelwerte je Cluster ===')
print(pd.concat([cluster_sizes, cluster_profile], axis=1).to_string())
```

```
=== Cluster-Profile: Mittelwerte je Cluster ===
         n_Kunden  CCreditScore   CBalance  CEstimatedSalary  CLV_Continuous   CA
ge  Account_Age_Months  Avg_Monthly_Transactions  Mobile_App_Usage_Hours  Last_Lo
gin_Days_Ago  CIsActiveMember  CNumOfProducts  Churn
Cluster
0            5136        653.02  121332.81         101579.31        17240.05  38.
09               13.55                     11.87                    5.04
16.15             0.54            1.37   0.09
1            3385        649.42    2265.71          99485.07         8265.19  37.
84               13.28                     11.94                    4.94
15.94             0.54            1.79   0.05
```

2) Cluster-Beschreibung und Profilnamen

Aus den Mittelwerten lassen sich die zwei Cluster folgendermaßen charakterisieren. Die Gesamtdurchschnitte des Datensatzes (Ø CBalance: 74.033 €, Ø CLV: 13.675 €, Ø Churn:

7,3 %, Ø Produkte: 1,54) dienen dabei als Referenz.

Cluster 0 — „Kapitalstarke Bestandskunden" (n=5.136, 60,3 % der Kunden):

- Deutlich überdurchschnittlicher Kontostand: Ø 121.333 € (Gesamtdurchschnitt: 74.033 €, Faktor: 53,6×gegenüber Cluster 1)
- CreditScore (653) und Gehalt (101.579 €) nahezu identisch mit dem Gesamtdurchschnitt (651,6 bzw. 100.747 €)
- Überdurchschnittlicher CLV: Ø 17.240 € – der hohe Kontostand schlägt sich direkt im Kundenwert nieder
- Überdurchschnittliche Churn-Rate: 9,0 % (vs. 7,3 % gesamt) – trotz hohem Kapital das risikoreichere Segment
- Weniger Produkte pro Kunde: Ø 1,37 (unter dem Gesamtdurchschnitt von 1,54)

Cluster 1 — „Kapitalarme Kunden mit Potenzial" (n=3.385, 39,7 % der Kunden):

- Sehr niedriger Kontostand: Ø 2.266 € – bei nahezu identischem Gehalt (99.485 € vs. 101.579 € in Cluster 0)
- Unterdurchschnittlicher CLV: Ø 8.265 €, was direkt aus dem niedrigen Kontostand resultiert
- Unterdurchschnittliche Churn-Rate: 4,8 % – das loyalere Segment, trotz niedrigerem Kapital
- Mehr Produkte pro Kunde: Ø 1,79 (über dem Gesamtdurchschnitt von 1,54) – breitere Produktnutzung

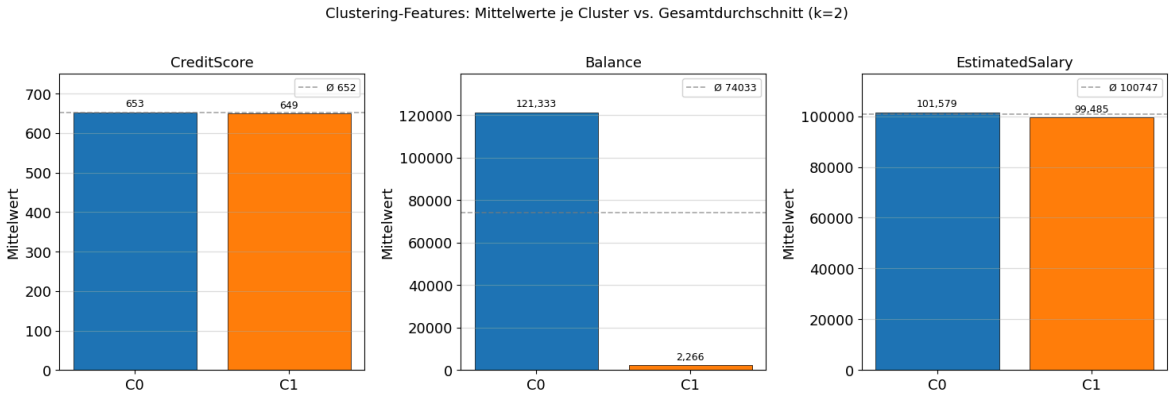
Auffällig: Die beiden Cluster unterscheiden sich bei fast identischem Gehalt und CreditScore primär im Kontostand – `CBalance` ist das dominierende Trennmerkmal. Das deckt sich mit dem Ergebnis der Vorwärts-Feature-Selektion, wo `CBalance` allein bereits einen Silhouette-Score von ~0,80 erzielte, sowie mit der Vergleichstabelle (Faktor 53,6×). Die App-Nutzung und der letzte Login sind zwischen beiden Clustern nahezu identisch (Cluster 0: 5,04 h, 16,15 Tage; Cluster 1: 4,94 h, 15,94 Tage), was zeigt, dass das digitale Engagement kein Unterscheidungsmerkmal ist.

3) Vergleichende Visualisierungen

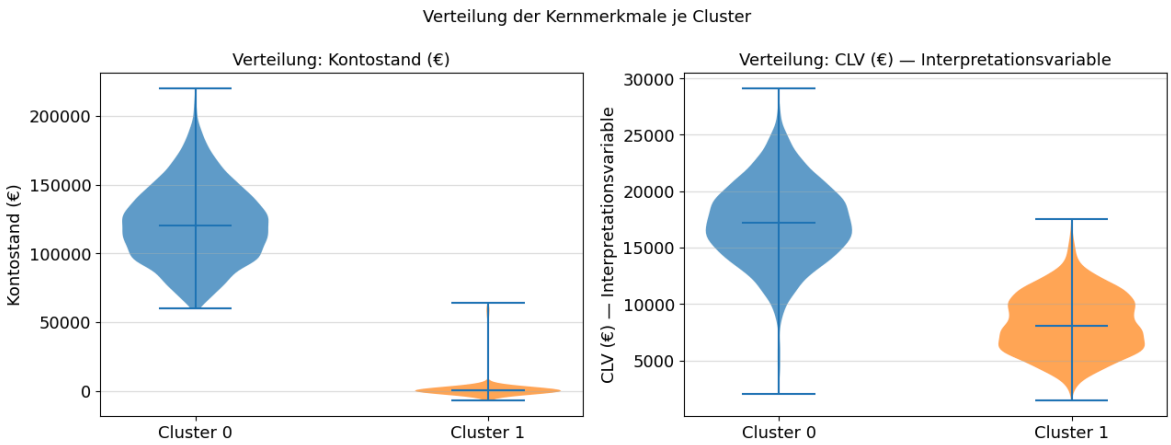
Die folgenden Plots vergleichen die Clustering-Features und zusätzliche Interpretationsvariablen je Cluster. Dabei werden die drei Clustering-Features (Balkendiagramm) und ergänzend `CLV_Continuous` (Violin-Plot) sowie weitere Merkmale (Heatmap) betrachtet. Die Heatmap macht mit einer Trennlinie transparent, welche Features die Cluster gebildet haben und welche nur zur Interpretation herangezogen werden.

In [16]:
```
# ── Bar-Vergleich: Clustering-Features je Cluster ──────────────────────────
fig, axes = plt.subplots(1, len(FINAL_FEATURES), figsize=(5 * len(FINAL_FEATURES
if len(FINAL_FEATURES) == 1:
    axes = [axes]

for i, feat in enumerate(FINAL_FEATURES):
    vals = cluster_profile[feat]
```

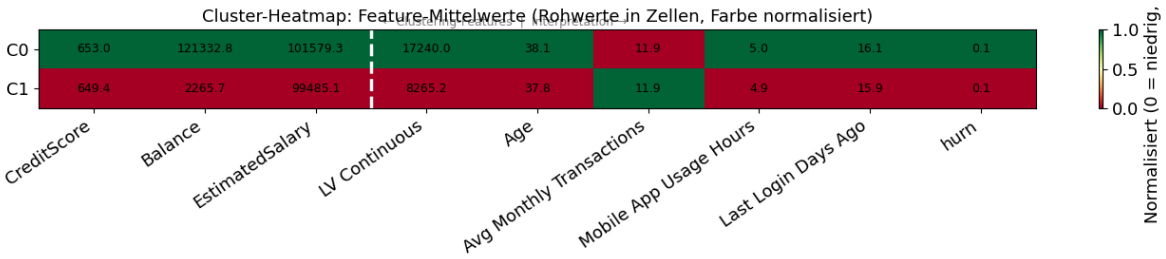
```
    bars = axes[i].bar(
        [f'C{c}' for c in vals.index], vals.values,
        color=[COLORS[c] for c in vals.index], edgecolor='black', linewidth=0.5
    )
    gm = df[feat].mean()
    axes[i].axhline(gm, color='gray', linestyle='--', linewidth=1.2,
                    alpha=0.7, label=f'Ø {gm:.0f}')
    axes[i].set_title(feat.replace('C','',1).replace('_',' '))
    axes[i].set_ylabel('Mittelwert'); axes[i].legend(fontsize=9)
    axes[i].grid(axis='y', alpha=0.4)
    y_top = max(vals.values.max(), gm) * 1.15
    axes[i].set_ylim(0, y_top)
    for bar, val in zip(bars, vals.values):
        axes[i].text(bar.get_x() + bar.get_width()/2, val + y_top*0.01,
                     f'{val:,.0f}', ha='center', va='bottom', fontsize=9)

plt.suptitle(f'Clustering-Features: Mittelwerte je Cluster vs. Gesamtdurchschnit
             fontsize=13, y=1.01)
plt.tight_layout()
plt.show()
```



In [17]:
```
# ── Violin-Plots: Verteilung CBalance und CLV (Interpretationsvariable) ──────
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, feat, label in [
    (axes[0], 'CBalance',       'Kontostand (€)'),
    (axes[1], 'CLV_Continuous', 'CLV (€) — Interpretationsvariable'),
]:
    data  = [df[df['Cluster']==c][feat].values for c in range(OPTIMAL_K)]
    parts = ax.violinplot(data, positions=range(OPTIMAL_K), showmedians=True)
    for i, pc in enumerate(parts['bodies']):
        pc.set_facecolor(COLORS[i]); pc.set_alpha(0.7)
    ax.set_xticks(range(OPTIMAL_K))
    ax.set_xticklabels([f'Cluster {c}' for c in range(OPTIMAL_K)])
    ax.set_ylabel(label); ax.set_title(f'Verteilung: {label}')
    ax.grid(axis='y', alpha=0.4)

plt.suptitle('Verteilung der Kernmerkmale je Cluster', fontsize=13)
plt.tight_layout(); plt.show()
```



In [18]:
```
# ── Heatmap: Normalisierte Feature-Mittelwerte ──────────────────────────
heatmap_cols = FINAL_FEATURES + ['CLV_Continuous', 'CAge', 'Avg_Monthly_Transact
                                 'Mobile_App_Usage_Hours', 'Last_Login_Days_Ago
CLUSTER_NAMES = {c: f'C{c}' for c in range(OPTIMAL_K)}

profile_heat = cluster_profile[heatmap_cols].copy()
profile_norm = (profile_heat - profile_heat.min()) / (profile_heat.max() - profi

fig, ax = plt.subplots(figsize=(14, max(3, OPTIMAL_K * 1.5)))
im = ax.imshow(profile_norm.values, aspect='auto', cmap='RdYlGn')
ax.set_xticks(range(len(heatmap_cols)))
ax.set_xticklabels([f.replace('C','',1).replace('_',' ') for f in heatmap_cols],
                   rotation=35, ha='right')
ax.set_yticks(range(OPTIMAL_K))
ax.set_yticklabels([CLUSTER_NAMES[c] for c in range(OPTIMAL_K)])
for i in range(OPTIMAL_K):
    for j, feat in enumerate(heatmap_cols):
        ax.text(j, i, f'{profile_heat.iloc[i,j]:.1f}', ha='center', va='center',
plt.colorbar(im, ax=ax, label='Normalisiert (0 = niedrig, 1 = hoch)')
# Trennlinie zwischen Clustering-Features und Interpretationsvariablen
ax.axvline(len(FINAL_FEATURES) - 0.5, color='white', linewidth=2.5, linestyle='-
ax.text(len(FINAL_FEATURES) - 0.4, -0.6, '← Clustering-Features  |  Interpretati
        ha='left', fontsize=9, color='gray')
ax.set_title('Cluster-Heatmap: Feature-Mittelwerte (Rohwerte in Zellen, Farbe no
plt.tight_layout(); plt.show()
```



Interpretation Visualisierungen:

Das Balkendiagramm bestätigt visuell, was die Profilmittelwerte numerisch zeigen: `CBalance` ist das einzige Feature, das tatsächlich zwischen den Clustern trennt – Cluster 0 liegt bei 121.333 €, Cluster 1 bei 2.266 €. Der gestrichelte Gesamtdurchschnitt (74.033 €) macht deutlich, dass Cluster 0 weit darüber, Cluster 1 weit darunter liegt. `CCreditScore` (653 vs. 649) und `CEstimatedSalary` (101.579 € vs. 99.485 €) zeigen dagegen kaum Unterschiede – die Balken liegen nahezu auf gleicher Höhe und beide nahe am Gesamtdurchschnitt.

Die Violin-Plots zeigen die Verteilung von `CBalance` und `CLV_Continuous` je Cluster. Beim Kontostand ist die Trennung extrem ausgeprägt: Der gesamte Violin von Cluster 0 liegt weit über dem von Cluster 1. Beim CLV zeigt Cluster 0 eine rechtsgezogene Verteilung mit höherem Median. Dass der CLV – obwohl er die Clusterbildung nicht beeinflusst hat (er wurde in Aufgabe 3.1 ausgeschlossen) – so deutlich zwischen den Clustern variiert, ist kein Zufall: Der CLV wurde in Assignment 2 zu einem wesentlichen Teil aus `CBalance` abgeleitet, was die enge Korrelation erklärt.

Die Heatmap fasst alle Merkmale normalisiert zusammen. Die gestrichelte Trennlinie macht transparent, welche Features das Clustering gesteuert haben (links: `CCreditScore`, `CBalance`, `CEstimatedSalary`) und welche nur zur Interpretation dienen (rechts). In der Heatmap ist das Muster eindeutig: Nur `CBalance` und der daraus abgeleitete `CLV_Continuous` zeigen einen deutlichen Farbunterschied zwischen den zwei Zeilen.

4) Unterschiede und Gemeinsamkeiten der Cluster

Die folgende Tabelle quantifiziert die Unterschiede zwischen den beiden Clustern. Der Faktor für `CBalance` beträgt 53,6 – das bestätigt den Kontostand als mit Abstand dominante Trennlinie. `CCreditScore` (Δ = 4 Punkte, Faktor 1,0×) und `CEstimatedSalary` (Δ = 2.094 €, Faktor 1,0×) unterscheiden sich dagegen praktisch nicht – die beiden Segmente trennen sich ausschließlich über das gespeicherte Kapital, nicht über Bonität oder Einkommen.

In [19]:
```
# ── Profilübersicht: Vergleichstabelle ──────────────────────────────
summary_cols = ['n_Kunden'] + profile_cols
summary_df   = pd.concat([cluster_sizes, cluster_profile], axis=1)

print('=== Cluster-Vergleich: Wichtigste Merkmale ===')
print()

# Differenz zwischen Clustern für die Clustering-Features
print('Relative Unterschiede in den Clustering-Features (Verhältnis max/min):')
for feat in FINAL_FEATURES:
    vals    = cluster_profile[feat]
    ratio   = vals.max() / vals.min() if vals.min() > 0 else float('nan')
    diff    = vals.max() - vals.min()
    print(f'  {feat:<25}  Δ = {diff:>10,.0f}   Faktor: {ratio:.1f}x')

print()
print('Churn-Rate je Cluster:')
for c in range(OPTIMAL_K):
    print(f'  Cluster {c}: {cluster_profile.loc[c, "Churn"]*100:.1f}%')
```

```
=== Cluster-Vergleich: Wichtigste Merkmale ===

Relative Unterschiede in den Clustering-Features (Verhältnis max/min):
  CCreditScore              Δ =          4   Faktor: 1.0x
  CBalance                  Δ =    119,067   Faktor: 53.6x
  CEstimatedSalary          Δ =      2,094   Faktor: 1.0x

Churn-Rate je Cluster:
  Cluster 0: 9.0%
  Cluster 1: 5.0%
```

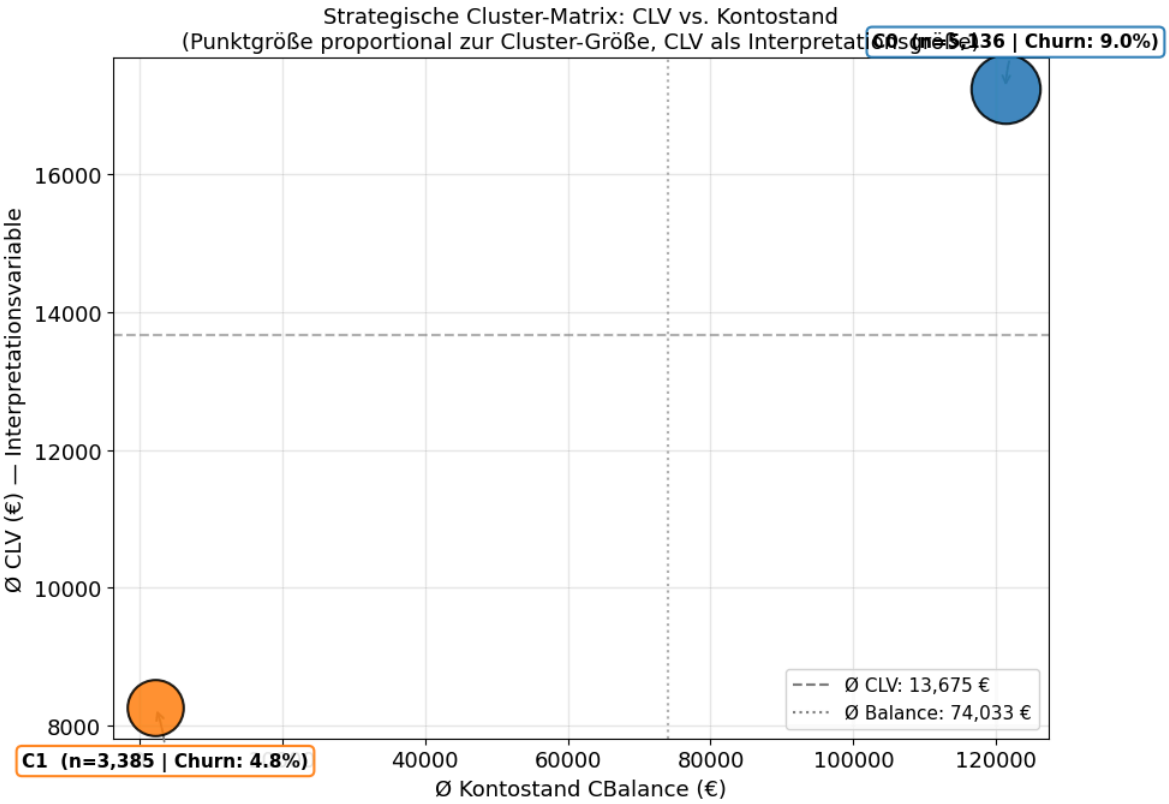
Aufgabe 3.4: Marketing-Empfehlungen

Aus den Cluster-Profilen lassen sich jetzt konkrete Maßnahmen ableiten. Die Grundidee: Die zwei identifizierten Segmente unterscheiden sich primär im Kontostand – bei vergleichbarem Gehalt und CreditScore. Das bedeutet: Die passenden Produkte und Kommunikationskanäle müssen an der Frage ansetzen, ob ein Kunde sein Kapital bereits bei der Bank platziert hat oder nicht.

In [20]:

```python
# ── Strategische Positionierungsmatrix: CLV vs. Kontostand ──────────────
# CLV_Continuous wird hier als Interpretationsgröße für die Visualisierung genut
fig, ax = plt.subplots(figsize=(10, 7))

clv_global = df['CLV_Continuous'].mean()
bal_global = df['CBalance'].mean()

cluster_data = {}
for c in range(OPTIMAL_K):
    grp = df[df['Cluster'] == c]
    cluster_data[c] = {
        'bal':   grp['CBalance'].mean(),
        'clv':   grp['CLV_Continuous'].mean(),
        'n':     len(grp),
        'churn': grp['Churn'].mean()
    }

for c in range(OPTIMAL_K):
    d = cluster_data[c]
    ax.scatter(d['bal'], d['clv'], s=d['n']/3,
               color=COLORS[c], alpha=0.85,
               edgecolors='black', linewidth=1.5, zorder=5)

for c in range(OPTIMAL_K):
    d = cluster_data[c]
    ax.annotate(
        f'C{c}  (n={d["n"]:,} | Churn: {d["churn"]*100:.1f}%)',
        (d['bal'], d['clv']),
        textcoords='offset points',
        xytext=(-80, 25 if c % 2 == 0 else -35),
        ha='left',
        fontsize=11, fontweight='bold', color='black',
        bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
                  edgecolor=COLORS[c], linewidth=1.5, alpha=0.9),
        arrowprops=dict(arrowstyle='->', color='gray',
                        lw=1.2, connectionstyle='arc3, rad=0.1')
    )
```

```
ax.axhline(clv_global, color='gray', linestyle='--', alpha=0.6)
ax.axvline(bal_global, color='gray', linestyle=':', alpha=0.6)

handles = [
    Line2D([0],[0], color='gray', linestyle='--', label=f'Ø CLV: {clv_global:,.0
    Line2D([0],[0], color='gray', linestyle=':',  label=f'Ø Balance: {bal_global
]
ax.legend(handles=handles, fontsize=11, loc='lower right')
ax.set_xlabel('Ø Kontostand CBalance (€)')
ax.set_ylabel('Ø CLV (€) — Interpretationsvariable')
ax.set_title('Strategische Cluster-Matrix: CLV vs. Kontostand\n'
             '(Punktgröße proportional zur Cluster-Größe, CLV als Interpretation
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



1) Segment-spezifische Handlungsempfehlungen

Die Marketing-Strategien leiten sich direkt aus den beiden identifizierten Kundensegmenten ab. Die zentrale Trennlinie ist der Kontostand: Cluster 0 (kapitalstark, 60,3 % der Kunden) hat einen Ø-Kontostand von 121.333 €, Cluster 1 (kapitalarm, 39,7 %) liegt bei nur 2.266 € – bei einem nahezu identischen Durchschnittsgehalt (101.579 € vs. 99.485 €). Die strategische Positionierungsmatrix visualisiert zusätzlich, wie sich CLV und Churn-Rate zwischen den Segmenten verhalten.

In [21]:
```
# ── Automatische Cluster-Charakterisierung für Marketing-Ableitung ──────────
# Klassifizierung der Cluster nach Kontostand und Churn-Rate für Strategiegerüst

bal_mean_global   = df['CBalance'].mean()
churn_mean_global = df['Churn'].mean()
```

```
clv_mean_global   = df['CLV_Continuous'].mean()

print('=== Strategische Einordnung der Cluster ===')
print(f'Globale Referenz → Ø CBalance: {bal_mean_global:,.0f} € | '
      f'Ø Churn: {churn_mean_global*100:.1f}% | Ø CLV: {clv_mean_global:,.0f} €'
print()

for c in range(OPTIMAL_K):
    d    = cluster_data[c]
    bal_label   = 'kapitalstark' if d['bal']   > bal_mean_global   else 'kapital
    churn_label = 'hohes Churn-Risiko' if d['churn'] > churn_mean_global else 'n
    clv_label   = 'hoher CLV' if d['clv'] > clv_mean_global else 'niedriger CLV'
    print(f'Cluster {c} (n={d["n"]:,}): {bal_label} | {churn_label} | {clv_label
    print(f'  Ø Balance: {d["bal"]:>10,.0f} € | Churn: {d["churn"]*100:.1f}% | C
    print()
```

```
=== Strategische Einordnung der Cluster ===
Globale Referenz → Ø CBalance: 74,033 € | Ø Churn: 7.3% | Ø CLV: 13,675 €

Cluster 0 (n=5,136): kapitalstark | hohes Churn-Risiko | hoher CLV
  Ø Balance:    121,333 € | Churn: 9.0% | CLV: 17,240 €

Cluster 1 (n=3,385): kapitalarm | niedriges Churn-Risiko | niedriger CLV
  Ø Balance:      2,266 € | Churn: 4.8% | CLV: 8,265 €
```

2) Marketing-Strategien je Segment

Cluster 0 — Kapitalstarke Bestandskunden (n=5.136, Ø Balance 121.333 €, Churn 9,0 %):
Marketingziel: Premium-Bindung & Churn-Prävention

Cluster 0 ist das wirtschaftlich wertvollste Segment mit einem durchschnittlichen CLV von 17.240 € – deutlich über dem Gesamtdurchschnitt von 13.675 €. Das überraschende Befund: Gerade diese kapitalstarken Kunden weisen mit 9,0 % die höhere Churn-Rate auf (Gesamtdurchschnitt: 7,3 %). Mit einem Gehalt von 101.579 € und einem CreditScore von 653 – beide nahezu identisch mit Cluster 1 – ist der hohe Kontostand das alleinige Differenzierungsmerkmal. Wer viel Kapital bei der Bank hält, vergleicht offenbar aktiver Konditionen. Jeder verlorene Kunde bedeutet hier einen überdurchschnittlichen CLV-Verlust, deshalb hat Retention oberste Priorität.

- **Strategie 1: Proaktive Churn-Prävention**
  Das Churn-Klassifikationsmodell aus Assignment 1 lässt sich direkt auf dieses Segment anwenden, um Abwanderungsrisiken frühzeitig zu erkennen – womit alle drei Assignments sinnvoll verknüpft werden.
  → Kanal: Persönliche E-Mail oder dedizierter Kundenbetreuer
  → Inhalt: Individuelle Konditionenangebote, Zufriedenheitsbefragungen vor Vertragsverlängerungen

- **Strategie 2: Cross-Selling von Premium-Anlageprodukten**
  Hohe Kontostände signalisieren Kapitalanlagebereitschaft. Bemerkenswert: Cluster 0 nutzt mit Ø 1,37 Produkten weniger als der Gesamtdurchschnitt (1,54) – trotz

höherem Kapital. Hier liegt ungenutztes Cross-Selling-Potenzial.
→ Kanal: In-App „Exklusive Angebote", persönliche Finanzberatung
→ Inhalt: ETF-Sparpläne, Festgeldangebote, Early-Access zu neuen Produkten, bevorzugte Kreditkonditionen

---

Cluster 1 — Kapitalarme Kunden mit Potenzial (n=3.385, Ø Balance 2.266 €, Churn 4,8 %):
Marketingziel: Kapitalaktivierung & Bindungsausbau

Diese Kunden haben trotz eines Durchschnittsgehalts von 99.485 € – nahezu identisch mit Cluster 0 (101.579 €) – kaum Kapital bei der Bank. Das ist kein Loyalitätsproblem: Die Churn-Rate liegt mit 4,8 % deutlich unter dem Gesamtdurchschnitt (7,3 %) und ist das niedrigste Segment. Es handelt sich um ein **Aktivierungsproblem**: Das Potenzial ist vorhanden, wird aber nicht ausgeschöpft. Positiv zu vermerken: Cluster 1 nutzt mit Ø 1,79 Produkten mehr als Cluster 0 (1,37) und über dem Gesamtdurchschnitt (1,54) – eine breite Produktbasis ist bereits vorhanden.

- **Strategie 1: Sparprodukt-Kampagnen zur Kapitalaktivierung**
  Ziel ist der schrittweise Aufbau des Kontostands durch niedrigschwellige Einstiegsangebote. Kein aggressives Upselling – das passt nicht zu einem Segment, das trotz hohem Einkommen kaum Kapital bei der Bank hält und möglicherweise anderswo spart.
  → Kanal: E-Mail, In-App-Benachrichtigung
  → Inhalt: Starter-Sparpläne, Ersteinlagen-Boni, automatisiertes Runden-auf-Sparen (Roundup-Feature)

- **Strategie 2: Digitale Vertiefung der Produktnutzung**
  Die App-Nutzung ist mit Ø 4,94 Stunden nahezu identisch mit Cluster 0 (5,04 Stunden) – der digitale Kanal funktioniert und die Bereitschaft zur Produktnutzung ist vorhanden (höhere Produktanzahl). Was fehlt, ist der Anreiz zur Kapitalkonzentration bei der Bank.
  → Kanal: In-App (Spar-Ziele, Fortschrittsanzeigen, Gamification), Push-Nachrichten
  → Inhalt: Spar-Challenges, gezielte Produktempfehlungen basierend auf bestehender Nutzung, Loyalitätsprämien

In [22]:
```
# ── Übersichtstabelle: Cluster-Strategie ──────────────────────────────
strategy_rows = []
for c in range(OPTIMAL_K):
    d = cluster_data[c]
    if d['bal'] > bal_mean_global:
        strategie  = 'Premium-Bindung & Churn-Prävention'
        massnahmen = 'Churn-Prävention, Premium-Anlageprodukte, VIP-Kommunikatio
    else:
        strategie  = 'Aktivierung & Potenzialentfaltung'
        massnahmen = 'Sparprodukt-Kampagnen, digitale Aktivierung, Cross-Selling
    strategy_rows.append({
        'Cluster':    c,
        'Kunden':     d['n'],
        'Ø Balance':  f"{d['bal']:,.0f} €",
```

```
        'Ø CLV':       f"{d['clv']:,.0f} €",
        'Churn':       f"{d['churn']*100:.1f}%",
        'Strategie':   strategie,
        'Maßnahmen':   massnahmen,
    })

strategy_df = pd.DataFrame(strategy_rows).set_index('Cluster')
print('=== Marketing-Strategien je Kundensegment ===')
print(strategy_df.to_string())
```

```
=== Marketing-Strategien je Kundensegment ===
         Kunden  Ø Balance     Ø CLV Churn                          Strategie
Maßnahmen
Cluster
0          5136  121,333 €  17,240 €  9.0%  Premium-Bindung & Churn-Prävention  C
hurn-Prävention, Premium-Anlageprodukte, VIP-Kommunikation
1          3385    2,266 €   8,265 €  4.8%   Aktivierung & Potenzialentfaltung
Sparprodukt-Kampagnen, digitale Aktivierung, Cross-Selling
```